

Adversarial Detection: Attacking Object Detection in Real Time

1st Han Wu
Computer Science
The University of Exeter
Exeter, the United Kingdom
hw630@exeter.ac.uk

2nd Syed Yunas
Computer Science
The University of the West of England
Bristol, the United Kingdom
syed.yunas@uwe.ac.uk

3rd Sareh Rowlands
Computer Science
The University of Exeter
Exeter, the United Kingdom
s.rowlands@exeter.ac.uk

4th Wenjie Ruan
Computer Science
The University of Exeter
Exeter, the United Kingdom
w.ruan@exeter.ac.uk

5th Johan Wahlström*
Computer Science
The University of Exeter
Exeter, the United Kingdom
j.wahlstrom@exeter.ac.uk

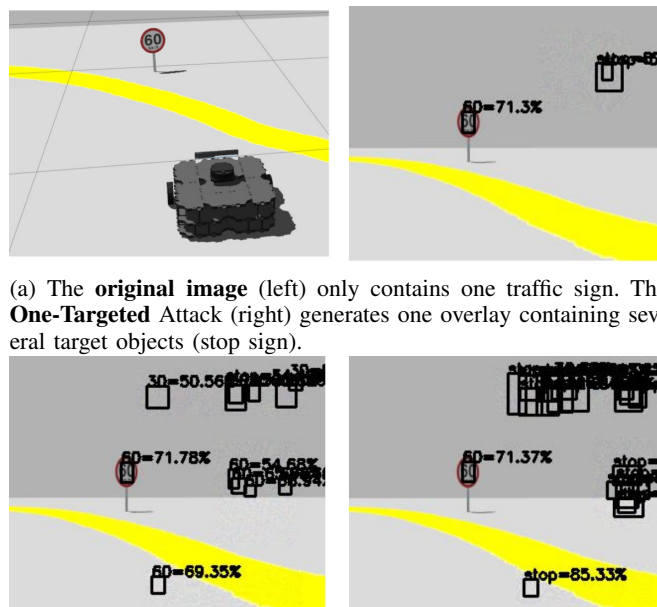
Abstract—Intelligent robots rely on object detection models to perceive the environment. Following advances in deep learning security it has been revealed that object detection models are vulnerable to adversarial attacks. However, prior research primarily focuses on attacking static images or offline videos. Therefore, it is still unclear if such attacks could jeopardize real-world robotic applications in dynamic environments. This paper bridges this gap by presenting the first real-time online attack against object detection models. We devise three attacks that fabricate bounding boxes for nonexistent objects at desired locations. The attacks achieve a success rate of about 90% within about 20 iterations. The demo video is available at <https://youtu.be/zJZ1aNIXsMU>.

Index Terms—Adversarial Attacks, Object Detection.

I. INTRODUCTION

Reliable object detection is crucial for multiple safety-critical robotic applications. For example, an autonomous driving system relies on object detection models to perceive the environment and take action on it. While advances in deep neural networks have rapidly increased the availability of high-accuracy object detection models, this breakthrough has also revealed several potential vulnerabilities. The first adversarial attacks fooled image classification models by adding imperceptible perturbations to the input image [1]. Later research removed the restriction to imperceptible perturbations and instead designed an adversarial patch that can be printed in the physical world [2]. The physical patch fooled an image classification model to misclassify the most salient object in the image. However, attacks against image classification models cannot fool object detection models. Therefore, in 2018, Liu et al. introduced the DPatch, a digital patch that fools object detection models [3], and Lee et al. later extended the attack to physical patches [4].

The project is supported by Offshore Robotics for Certification of Assets (ORCA) Partnership Resource Fund (PRF) on Towards the Accountable and Explainable Learning-enabled Autonomous Robotic Systems (AELARS) [EP/R026173/1].



(a) The **original image** (left) only contains one traffic sign. The **One-Targeted Attack** (right) generates one overlay containing several target objects (stop sign).

(b) The **Multi-Untargeted Attack** (left) generates multiple overlays containing different kinds of objects (30, 60, stop). The **Multi-Targeted Attack** (right) generates multiple overlays containing several target objects (stop sign).

Fig. 1: This paper demonstrates how to generate adversarial overlays of arbitrary shapes at specified positions in real time.

The generation of adversarial patches for attacking real-time robotic applications is still a rather challenging task. The generation process itself is often very computationally expensive. In addition, the efficiency of the physical patch is conditioned on strict requirements on the relative distance and orientation of the camera and the patch [5] [6]. These requirements are often difficult to satisfy for real-world robotic applications deployed in dynamic environments.

Recent research in penetration tests against the Robot Operating System (ROS) has shown that it is possible to inject digital patches into the ROS message that contains the camera image [7]. In particular, Dieber et al. succeeded in isolating a ROS node so that they could intercept and manipulate the ROS message that contains sensor data while the victim node is unaware of the attack. In this paper, we generate digital overlays (unperceivable digital patches) at desired locations in real time and then exploit the vulnerability to inject the digital overlays into the input image. Overall, this paper makes the following contributions:

- We devise three adversarial attacks that can generate digital overlays of different shapes at specified locations in real time (see Fig. 1). This extends previous research that primarily has focused on square patches.
- Investigating the effect of various digital overlays it is found that the size of the overlay is critical for the success of the attack, whereas the attack performance is independent of the aspect ratio.
- We test our attacks in the Robot Operating System (ROS). The system is open-sourced to facilitate future extensions and comparisons¹.

II. PRELIMINARIES

This section clarifies the differences between adversarial filters and adversarial patches. Following this, we describe how prior research applies these filters and patches in the digital and physical world. *Our* research focuses on applying adversarial patches in the digital world.

A. The Adversarial Filter

An adversarial filter refers to a perturbation added to the entire image. Thus, the adversarial filter is of the same size as

¹The code is available on Github: <https://github.com/wuhanstudio/adversarial-detection>

the input image and is unperceivable by human eyes. Based on where we apply the perturbation, adversarial filters can be categorized as digital or physical filters.

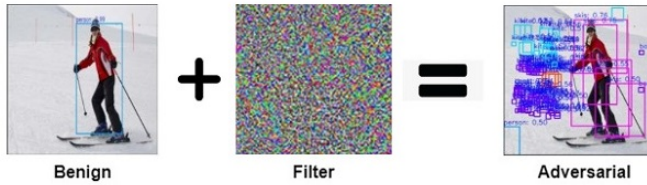
1) **Digital Filter** (Fig. 2a): The first adversarial attack against image classification models was a digital filter that added a small perturbation to the entire input image [1]. The perturbation can be generated using either gradient-based methods [9] [10] [11] [12] or optimization-based methods [13] [14] [15]. Prior research uses the l_1 , l_2 , and l_∞ norms to measure the human perceptual distance between the adversarial and the original input image [16] [17] [18]. While the digital filter first proved the existence of adversarial examples, it is limited in practical scenarios since it is not always possible to assume access to the input image.

2) **Physical Filter** (Fig. 2b): Physical filters attach a translucent film to the camera lens to perturb the entire image [8]. While physical filters do not require access to the input image, they do require physical access to the camera. One big challenge with physical filters is the difficulty of manufacturing a film that precisely replicates the adversarial perturbation. Thus, the practical applicability of adversarial filters in the physical world is still rather limited. In addition, to the best of our knowledge, physical filters has so far only been used to attack image classification models, not object detection models.

B. The Adversarial Patch

In some scenarios, it is possible to remove the constraint of the imperceptibility of the perturbation. This means that we can generate so-called patches that are clearly distinguishable to anyone viewing the image in question.

Contrary to filters, these patches are limited in size and only perturb part of the image, allowing us to better control the location of generated objects when attacking an object detection model. Similar to filters, patches can be categorized as digital or physical patches.



(a) Digital filters apply the perturbation to the entire input image in the digital world [5].



(b) Physical filters apply the perturbation by attacking the camera using, e.g., a translucent film attached to the camera lens [8].

Fig. 2: Adversarial filters perturb the entire image.



(a) Digital patches replace a part of the input image with the adversarial patch (left top corner) [3].



(b) Physical patches print the adversarial patch on a physical object, e.g., on a poster [4].

Fig. 3: Adversarial patches perturb part of the image.

1) **Digital Patch** (Fig. 3a): In 2018, Liu et al. designed the DPatch [3] to attack the Faster R-CNN object detection model [19]. The DPatch is location-independent, which means that it can be placed anywhere in the input image. Another notable digital patch is the TOG attack, presented in 2020 by Chow et al [20]. They generated a 40x40 adversarial patch to fool object detection models into misclassifying objects to which the patch was attached.

2) **Physical Patch** (Fig. 3b): In 2017, Google presented landmark research for generating a physical patch to attack an image classification model [2]. As a result, research interest gradually shifted to from digital to physical patches. Later, Lee et al. [4] and Wang et al. [5] extended the physical patch attack from image classification to object detection. One shortcoming of physical patches is their inflexibility. Once the adversarial patch has been printed out, the only way to change it is through reprinting. To improve their flexibility, Hoory et al. dynamically displayed adversarial patches on a flat screen attached to a car [21].

III. ADVERSARIAL DETECTION

Our objective is to fool the YOLOv3 [22], and YOLOv4 [23] object detection models to mistakenly detect objects at locations where there is no object. We generate adversarial overlays by combining the imperceptibility of adversarial filters with the localizability of adversarial patches.

A. Problem Formulation

The YOLO object detection model splits the input image into $S \times S$ grids and makes predictions for each grid. For example, the input shape of YOLO is 416x416x3 (height, width, and channel). If $S = 13$, we divide each channel into 13x13 grids, and each grid is 32x32 pixels. To detect objects of different sizes, YOLO makes predictions at three different scales (see Fig. 4). The first, second, and third output layer contains 13x13, 26x26, and 52x52 grids, respectively. The first output layer detects larger objects, and the third output layer detects smaller objects. In addition, YOLO pre-defines three anchor boxes ($B = 3$) at each scale to detect objects of different aspect ratios. Thus, we have 9 pre-defined anchor boxes for 3 scales. Lastly, each output contains the shape and location of the bounding box, the confidence value, and the probability vector for each class (see Fig. 4). For example, if the model is pretrained on the MS COCO dataset [24] [25] that contains 80 classes ($K = 80$), each output contains 85 values consisting of four dimensions (b_x, b_y, b_w, b_h), one confidence value (c), and 80 probabilities ($p_1, p_2, \dots, p_{80}$) for each class.

Putting things together: Given an input image x , the object detection model outputs $S \times S$ candidate bounding boxes $o \in \mathcal{O}$ at three different scales ($S \in \{13, 26, 52\}$, $B = 3$, and $|\mathcal{O}| = \sum_{1 \leq i \leq 3} S_i \times S_i \times B$, where S_i represents the grid size of the i_{th} output layer). Further, each candidate box is

$$o^i = [b_x^i, b_y^i, b_w^i, b_h^i, c^i, p_1^i, p_2^i, \dots, p_K^i], \quad 1 \leq i \leq |\mathcal{O}| \quad (1)$$

for K classes. For example, the shape of the first output layer is $(S_1, S_1, B, K + 5) = (13, 13, 3, 85)$. The image is

divided into 13x13 grids, and the model makes predictions for 3 anchor boxes. Each prediction contains 85 values ($85 = 4 + 1 + 80$). The adversarial attack aims to generate an adversarial perturbation $\delta \in [-1, 1]^{whc}$ such that the adversarial output $\mathcal{O}(x')$ is different from the original output $\mathcal{O}(x)$.

B. The Adversarial Overlay

As described in Section II, adversarial filters are imperceptible to human eyes but do not allow us to control exactly where in an image that we fabricate objects. Adversarial patches, on the other hand, are pinpointable but conspicuous. We propose a new method, the adversarial overlay, that applies an imperceptible perturbation to part of an image. Thereby, the adversarial overlay combines the strengths of adversarial filters and patches. The perturbations applied in filters, patches, and overlays can be contrasted as

$$x'_{filter} = x + \delta, \quad (2a)$$

$$x'_{patch} = (1 - m) \odot x + m \odot \delta, \quad (2b)$$

$$x'_{overlay} = x + m \odot \delta. \quad (2c)$$

Here, the perturbation δ is applied via a binary mask $m \in \{0, 1\}^{wh}$, and $\odot$ is the Hadamard operator performing element-wise multiplication. As seen in equation (2), the adversarial filter x'_{filter} applies the perturbation to the entire image without the mask m , the adversarial patch x'_{patch} uses the mask to replace part of the image, and the adversarial overlay $x'_{overlay}$ applies the mask to the perturbation and adds it as an overlay to the image.

Now, note that the confidence value and the probability vector collectively determine whether or not a bounding box is drawn on the output image. Thus, by maximizing the product of confidence and probability vector, we can fabricate objects at desired locations. This leads us to propose three adversarial loss functions L_{adv} for generating perturbations in real time: one loss function for one-targeted attacks

$$\max_{1 \leq i \leq |\mathcal{O}|} [\sigma(c^i) * \sigma(p_t^i)], \quad (3)$$

one loss function for multi-targeted attacks

$$\sum_{i=1}^{|\mathcal{O}|} [\sigma(c^i) * \sigma(p_t^i)], \quad (4)$$

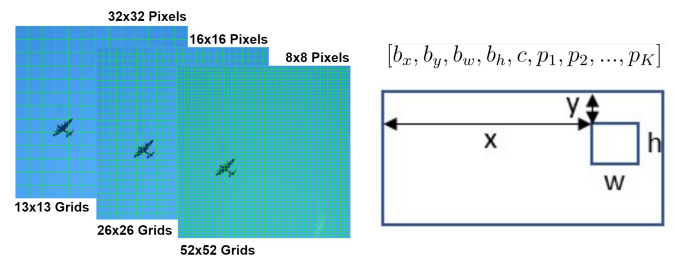


Fig. 4: The YOLO object detection model makes predictions at three scales.

and one loss function for multi-untargeted attack

$$\sum_{i=1}^{|\mathcal{O}|} \sum_{j=1}^K [\sigma(c^i) * \sigma(p_j^i)]. \quad (5)$$

Here, the function $\sigma(x)$ represents the sigmoid function.

The one-targeted attack (Eq. 3) generates one overlay that contains the target object $t \in \{1, \dots, K\}$ by finding the bounding box with the maximum value of $\sigma(c^i) * \sigma(p_t^i)$. This method generates only one overlay that contains target objects because we increase the confidence and probability of the maximum one in all candidate bounding boxes.

The multi-targeted attack (Eq. 4) generates multiple overlays that contain target objects $t \in [0, K]$ by maximizing the sum of $\sigma(c^i) * \sigma(p_t^i)$ for the target class.

The multi-untargeted attack (Eq. 5) generates multiple overlays that contain different objects by maximizing the sum of $\sigma(c^i) * \sigma(p_t^i)$ for all classes.

Note that it is also possible to generate monochrome greyscale overlays. Greyscale overlays add the same value to each channel, making the perturbation less conspicuous. We can generate monochrome overlays using the average of the red, green, and blue channels or from a single channel since human eyes are most sensitive to the green channel.

The perturbation is computed using gradients. At the end of each iteration, we clip the value of the perturbation so that it does not exceed the pre-defined boundary. The adversarial image of the original DPatch [3] contains invalid negative pixel values. Thus, we also clip the value of the adversarial image to make sure it is still a valid image. The adversarial overlay attack is summarized in Algorithm 1.

C. System Architecture

This paper presents an adversarial detection system to attack the YOLO object detection model. The system adopts a modular design pattern so that we can test our attacks in different environments (see Fig. 5) using the same architecture. The system consists of three components: the Data Source, the Server, and the Control Plane.

The Data Source: The data source publishes the input image to the server. We can publish the image from different sources, including a PC camera, the ROS Gazebo Simulator, and a real Turtlebot 3.

Algorithm 1 The Adversarial Overlay Attack

Input: The object detection model $f(\theta, x)$, a mask m .
Parameters: The number of iterations n , the learning rate α , and the strength of the attack ξ measured by l_∞ norm.
Output: The adversarial perturbation δ .

Initialization:

if monochrome **then**

$\delta \leftarrow 0^{416 \times 416}$

else

$\delta \leftarrow 0^{416 \times 416 \times 3}$

end if

for each input image x **do**

$x' = x$

for each iteration **do**

if monochrome **then**

R Channel: $x'_R = x'_R + m \odot \delta$

G Channel: $x'_G = x'_G + m \odot \delta$

B Channel: $x'_B = x'_B + m \odot \delta$

else

Overlay: $x' = x' + m \odot \delta$

end if

Gradient: $\nabla = \frac{\partial \mathcal{L}_{adv}(\mathcal{O})}{\partial x'}$

if monochrome **then**

$\delta = \delta + \frac{1}{3} \alpha (\nabla_R + \nabla_G + \nabla_B)$

else

$\delta = \delta + \alpha \cdot \text{sign}(\nabla)$

end if

$\delta = \text{clip}(\delta, -\xi, \xi)$

$x' = \text{clip}(x', 0.0, 1.0)$

end for

end for

The Server: The server receives the input image stream from the data source via WebSocket connections. Meanwhile, it obtains the adversarial mask from the control plane. The server then generates and injects the adversarial overlay into the input image. In addition, the YOLO object detection model is deployed on the server.

The Control Plane: The control plane is a website where the attacker can draw the mask at arbitrary locations. Then, the browser sends the mask to the server via WebSocket connections.

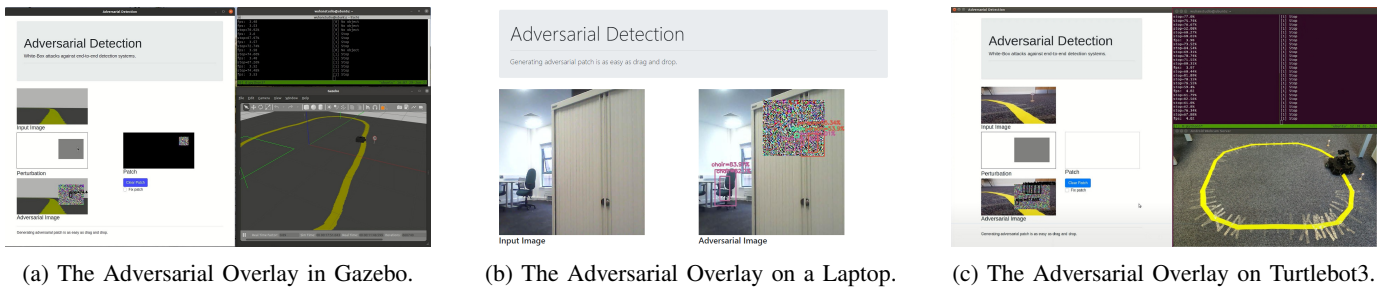
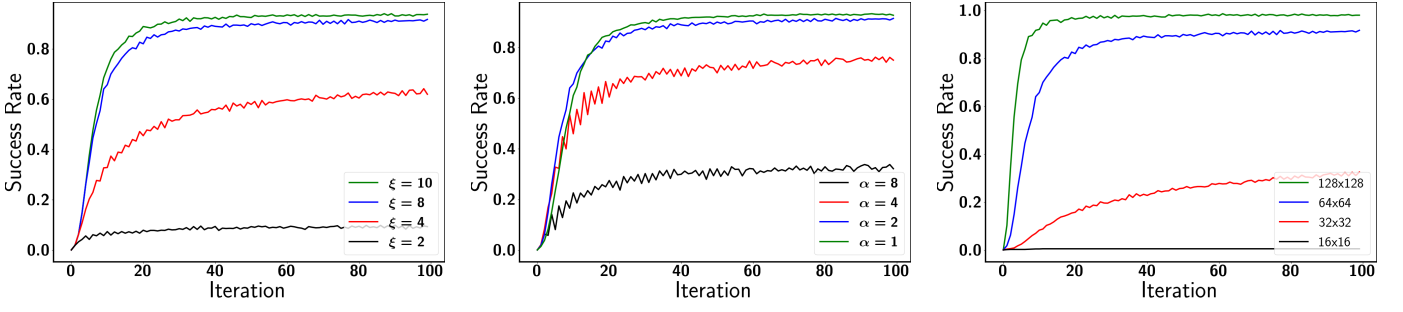
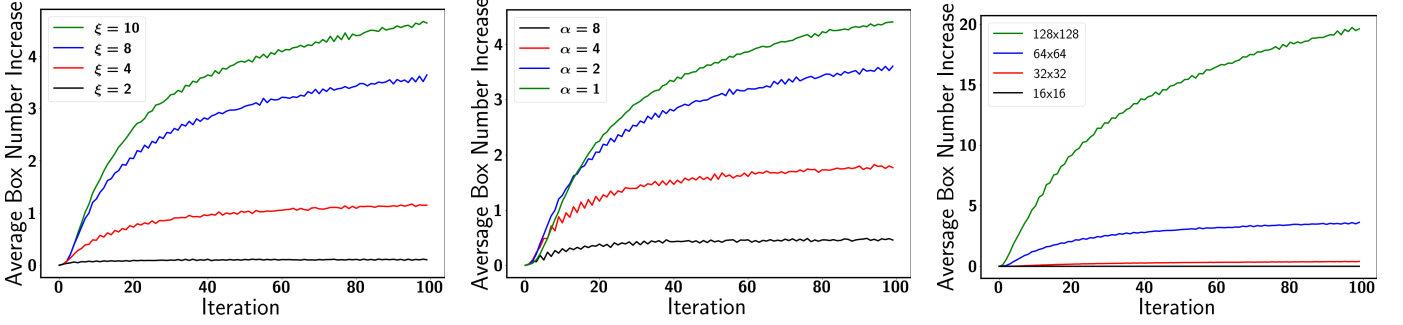


Fig. 5: We tested our attack in three environments. Here we made the overlay visible for illustration purposes.



(a) Different ξ with $\alpha = 2$, box size= 64. (b) Different α with $\xi = 8$, box size = 64. (c) Different box sizes with $\xi = 8, \alpha = 2$.

Fig. 6: The success rate of the multi-untargeted attack with different ξ , α , and box sizes.



(a) Different ξ with $\alpha = 2$, box size= 64. (b) Different α with $\xi = 8$, box size = 64. (c) Different box sizes with $\xi = 8, \alpha = 2$.

Fig. 7: The average box number increase of the multi-untargeted attack with different ξ , α , and box sizes

IV. EXPERIMENTAL RESULTS

We used YOLO object detection models pre-trained on the MS COCO dataset for the PC environment. Moreover, we trained two traffic sign detection models using YOLO for the ROS Gazebo and the ROS Turtlebot environment.

A. Evaluation Metrics

Prior research have used the Mean Average Precision (mAP) [26] to evaluate the accuracy of object detection models. However, the objective of our attack is to fabricate new bounding boxes, not to decrease the accuracy of existing objects. Therefore, the evaluation metrics need to reflect the efficiency of generating new bounding boxes in real time, something which mAP cannot do. In addition, the mAP evaluation metric requires access to the ground truth, that is, human-labeled bounding boxes. The online attack that fools the object detection system in real time does not have any such ground truth since it is impossible for humans to label the video stream in real time. Thus, we use the following two evaluation metrics that evaluate the efficiency of generating new objects and do not require ground truth labels.

1) **Success Rate:** The success rate measures the percentage of images we successfully fabricate at least one object within a limited number iterations. For a real-time online attack, we need to achieve a satisfying success rate within a limited number of iterations.

2) **Mean Box Number Increase:** We measure the number of new bounding boxes generated compared to the benign output. The more bounding boxes we generate, the stronger the perturbation is. However, to fabricate more objects we also need a larger number of iterations.

In general, we will attempt to find the minimum number of iterations required to achieve a satisfying success rate and fabricate enough bounding boxes. The PASCAL VOC-2012 validation set is used in the following experiments.

B. Hyper-parameters

Three hyper-parameters are crucial for the attack: the attack strength ξ , the step size α , and the box size. Since monochrome overlays can be generated in a less conspicuous way, we also investigated the effect of using gradients from different channels. In addition, we examined if the adversarial perturbation is sensitive to the aspect ratio.

1) **The Attack Strength:** The attack strength ξ sets a boundary on the maximum perturbation added to each pixel. A larger ξ makes the attack stronger but also results in a more conspicuous perturbation. In Fig. 6a, $\xi \in \{8, 10\}$ results in a much higher success rates than $\xi \in \{2, 4\}$ after 100 iterations. Though $\xi = 8$ and $\xi = 10$ share similar success rates, $\xi = 10$ generates more bounding boxes than $\xi = 8$ (see Fig. 7a). For a real-time attack, we aim to generate at least one bounding box in 30 iterations, a requirement which is satisfied with both $\xi = 8$ and $\xi = 10$. $\xi = 8$, in particular, finds a good

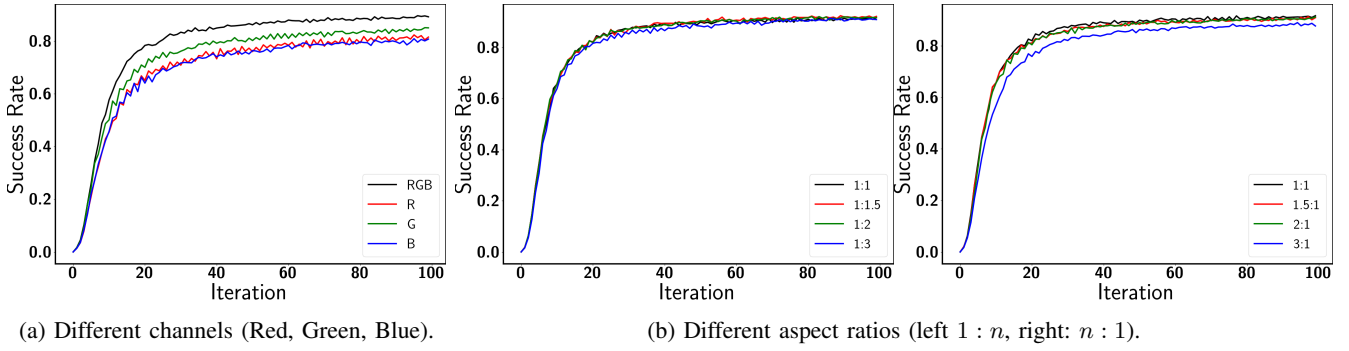


Fig. 8: The success rate of the multi-untargeted attack with different channels and aspect ratios.

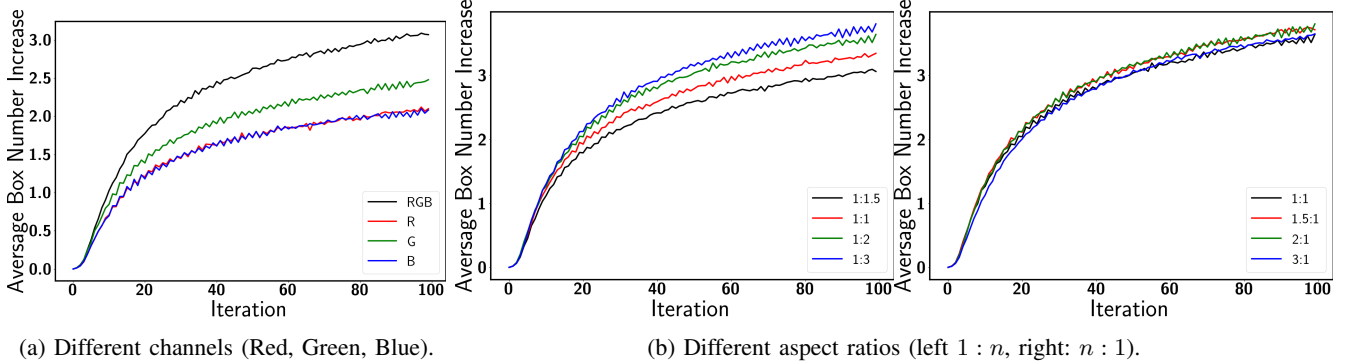


Fig. 9: The average box number increase of the multi-untargeted attack different channels and aspect ratios.

balance between the attack strength and the impermeability of the perturbation.

2) **The Step Size:** The step size α controls how fast we update the perturbation in each iteration. A larger α would make it possible to fabricate bounding boxes in fewer iterations but make the update more unstable. In Fig. 6b, both $\alpha = 1$ and $\alpha = 2$ achieve a success rate of more than 90% after just 30 iterations, whereas $\alpha > 4$ produces a worse the success rate. In Fig. 7b, $\alpha = 1$ eventually generates more bounding boxes than $\alpha = 2$, but for a real-time online attack, it's more important to have a fast attack.

3) **The Box Size:** The box size determines the total number of pixels we can perturb. The more pixels we can manipulate, the easier it is to deviate the model output. We need to find the minimum box size for our attack. In Fig. 6c, the box sizes of both 64x64 and 128x128 achieve a success rate of 90% after just 30 iterations, while the box size of 32x32 struggles to achieve successful attacks. Moreover, with box size smaller than 16x16 we are unable to attack the model with $\xi = 8$ and $\alpha = 2$. In addition, as expected, the larger the box size is, the more bounding boxes the attack can generate (see Fig. 7c). For a real-time online attack, we suggest the overlay box size to be at least 64x64.

4) **The Monochrome Channel:** For the attack that generates a polychrome overlay, we update each channel of the perturbation using the gradients of each channel. While for the monochrome overlay, we decide which color channel to prioritize. Interestingly, the experimental results (see Fig. 8a

and Fig. 9a) show that the object detection model is the most sensitive to the green channel. Using the average gradient of RGB channels, we achieve the highest success rate and generate the most number of boxes.

5) **The Aspect Ratio:** Our attack can generate adversarial overlays of arbitrary shapes. Thus, we can study if the object detection model is more susceptible to adversarial overlays of specific aspect ratios, e.g., wide (1:n) or long (n:1) boxes. For a fair comparison, overlay boxes of different aspect ratios have the same number of pixels perturbed. In Fig. 8b, different aspect ratios do not cause observable dissimilarities in success rates. Although the aspect ratio 1 : 3 generates more bounding boxes than 1 : 1.5 after 100 iterations (see Fig. 9b), this attributes to a slightly more number of boxes perturbed (1 : 3 $\rightarrow 37 \times 111 = 4108$, 1 : 1.5 $\rightarrow 52 \times 78 = 4056$). Thus, we cannot conclude that the aspect ratio has any discernible impact on the attack. As a result, we use $\xi = 8$, $\alpha = 2$, and a box size of 64x64 as default values for our attack, and the monochrome overlay uses the average of gradients from RGB channels.

C. The Attack Performance

We measured the performance of the attack on an NVIDIA RTX 2080Ti GPU. According to prior experimental results, we chose the hyper-parameters of $\xi = 8$, $\alpha = 2$, and used the box sizes of 64x64 and 128x128. Then, we tested the attack on the VOC 2012 validation set, which includes 5823 images in total (see Table I and II).

In Table I, the attack achieved 24 FPS (1 iteration cost 41 ms). It should be noted that the performance of the attack also depends on model size. A larger model requires more computations to compute the gradient. Besides, the time cost does not grow as the box size increases (64 x 64 and 128 x 128) since we calculate the gradient of the adversarial loss functions over the entire input image, making it possible to generate multiple overlays of different shapes simultaneously without recalculating gradients.

Box Sizes	1 iteration	10 iterations	20 iterations
64x64	41 ms	410 ms	780 ms
128x128	41 ms	410 ms	781 ms

TABLE I: The time cost of the attack with different numbers of iterations ($\alpha = 2$, $\xi = 8$).

Box Sizes	N=1	N=3	N=5
64x64	7.47 it (306 ms)	12.03 it (493 ms)	13.62 it (558 ms)
128x128	4.40 it (180 ms)	7.64 it (313 ms)	10.20 it (418 ms)

TABLE II: The average number of iterations and time cost of generating N bounding boxes ($\alpha = 2$, $\xi = 8$).

As illustrated in Table II, on average, the attack requires only 7 iterations within around 300 ms to generate one bounding box for each static image (offline attack).

For an online attack, the attack can be even more efficient. It is unnecessary to re-generate the adversarial overlay from scratch for every input video frame since there is a high correlation between consecutive video frames and iterations [27]. As a result, we can save computations by reusing the overlay generated in the previous timestep. In the demo video, the attack achieved near real-time performance on an Intel i7-8665U CPU.

V. CONCLUSIONS

This paper has demonstrated that it is possible to attack an object detection system in real time. We generate human unperceivable adversarial overlays of arbitrary shapes to fabricate bounding boxes at desired locations. This attack could be a threat to the areas of traffic sign recognition and the autonomous driving field as a whole.

In the future, we plan to investigate the effect of the attack on modular autonomous driving systems that rely on object detection models to perceive the environment. In addition, we will explore how to detect adversarial attacks so that we can embrace deep learning models in safety-critical robotic applications in a safe way.

REFERENCES

- [1] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," *arXiv preprint arXiv:1412.6572*, 2014.
- [2] T. Brown, D. Mane, A. Roy, M. Abadi, and J. Gilmer, "Adversarial patch," *arXiv preprint arXiv:1712.09665*, 2017.
- [3] X. Liu, H. Yang, Z. Liu, L. Song, H. Li, and Y. Chen, "Dpatch: An adversarial patch attack on object detectors," *arXiv preprint arXiv:1806.02299*, 2018.
- [4] M. Lee and Z. Kolter, "On physical adversarial patches for object detection," *arXiv preprint arXiv:1906.11897*, 2019.
- [5] D. Wang, C. Li, S. Wen, Q.-L. Han, S. Nepal, X. Zhang, and Y. Xiang, "Daedalus: Breaking nonmaximum suppression in object detection via adversarial examples," *IEEE Transactions on Cybernetics*, 2021.
- [6] M. Threet, C. Busho, J. Harguess, M. Jutras, N. Lape, S. Leary, K. Manville, M. Tan, and C. Ward, "Physical adversarial attacks in simulated environments," in *IEEE Applied Imagery Pattern Recognition Workshop (AIPR)*, 2021, pp. 1–5.
- [7] B. Dieber, R. White, S. Taurer, B. Breiling, G. Caiazza, H. Christensen, and A. Cortesi, "Penetration testing ros," in *Robot Operating System (ROS)*. Springer, 2020, pp. 183–225.
- [8] J. Li, F. Schmidt, and Z. Kolter, "Adversarial camera stickers: A physical camera-based attack on deep learning systems," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2019, pp. 3896–3904.
- [9] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2018.
- [10] A. Kurakin, I. J. Goodfellow, and S. Bengio, "Adversarial examples in the physical world," in *Artificial intelligence safety and security*. Chapman and Hall/CRC, 2018, pp. 99–112.
- [11] E. Wong, F. Schmidt, and Z. Kolter, "Wasserstein adversarial examples via projected sinkhorn iterations," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2019, pp. 6808–6817.
- [12] F. Croce and M. Hein, "Reliable evaluation of adversarial robustness with an ensemble of diverse parameter-free attacks," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2020, pp. 2206–2216.
- [13] N. Papernot, P. McDaniel, and I. Goodfellow, "Transferability in machine learning: from phenomena to black-box attacks using adversarial samples," *arXiv preprint arXiv:1605.07277*, 2016.
- [14] N. Carlini and D. Wagner, "Towards evaluating the robustness of neural networks," in *IEEE Symposium on Security and Privacy*, 2017, pp. 39–57.
- [15] Y. Qin, N. Carlini, G. Cottrell, I. Goodfellow, and C. Raffel, "Imperceptible, robust, and targeted adversarial examples for automatic speech recognition," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2019, pp. 5231–5240.
- [16] T. Miyato, S.-i. Maeda, M. Koyama, K. Nakae, and S. Ishii, "Distributional smoothing with virtual adversarial training," *arXiv preprint arXiv:1507.00677*, 2015.
- [17] S. Sabour, Y. Cao, F. Faghri, and D. J. Fleet, "Adversarial manipulation of deep representations," *arXiv preprint arXiv:1511.05122*, 2015.
- [18] P.-Y. Chen, Y. Sharma, H. Zhang, J. Yi, and C.-J. Hsieh, "Ead: elastic-net attacks to deep neural networks via adversarial examples," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 32, no. 1, 2018.
- [19] S. Ren, K. He, R. Girshick, and J. Sun, "Faster r-cnn: Towards real-time object detection with region proposal networks," *Advances in Neural Information Processing Systems*, vol. 28, 2015.
- [20] K.-H. Chow, L. Liu, M. Loper, J. Bae, M. E. Gursoy, S. Truex, W. Wei, and Y. Wu, "Adversarial objectness gradient attacks in real-time object detection systems," in *IEEE International Conference on Trust, Privacy and Security in Intelligent Systems and Applications (TPS-ISA)*, 2020, pp. 263–272.
- [21] S. Hoory, T. Shapira, A. Shabtai, and Y. Elovici, "Dynamic adversarial patch for evading object detection models," *arXiv preprint arXiv:2010.13070*, 2020.
- [22] J. Redmon and A. Farhadi, "Yolov3: An incremental improvement," *arXiv preprint arXiv:1804.02767*, 2018.
- [23] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "Yolov4: Optimal speed and accuracy of object detection," *arXiv preprint arXiv:2004.10934*, 2020.
- [24] T.-Y. Lin, M. Maire, S. Belongie, L. Bourdev, R. Girshick, J. Hays, P. Perona, D. Ramanan, C. L. Zitnick, and P. Dollár, "Microsoft coco: Common objects in context," 2014.
- [25] B. E. Moore and J. J. Corso, "Fiftyone," *GitHub. Note: https://github.com/voxel51/fiftyone*, 2020.
- [26] J. Cartucho, R. Ventura, and M. Veloso, "Robust object recognition through symbiotic deep learning in mobile robots," in *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2018, pp. 2336–2341.
- [27] A. Ilyas, L. Engstrom, and A. Madry, "Prior convictions: Black-box adversarial attacks with bandits and priors," *arXiv preprint arXiv:1807.07978*, 2018.